

Computer Programming: Lecture 6

Outline

- Functions
- Scope

Void functions

- Until now we have been writing void functions.
- Void functions doesn't return value to their caller.
- If you try to assign the result to a variable, you get a special value called None.

```
def greeting(name):  
    return 'hi ' + name
```

```
print(greeting('Haven'))  
value = greeting('Robel')  
print(value)
```

```
def maximum(x,y):  
    if(x > y):  
        print(x)  
    else:  
        print(y)  
maximum(2,4)
```

Fruitful functions

- A function that returns a value to its caller.
- When a return statement executes, the function terminates without executing any subsequent statements. The code that could not be reached by the flow execution is called dead code.

```
def greeting(name):  
    print('hi ',name)
```

```
greeting('Haven')  
greeting('Robel')
```

```
def maximum(x,y):  
    if(x > y):  
        return x  
    else:  
        return y  
    print("Hello")  
print(maximum(2,4))
```

```
def maximum(x,y,z):  
    if(x > y and x > z):  
        return x  
    elif(y > x and y > z):  
        return y  
    else:  
        return z
```

Scope

- A variable can have something called a 'scope'. This refers to its accessibility.
- There are two types of scopes:
 - Global: accessible from everywhere.
 - Local: only accessible within a function

Scope

```
name = "Hello" #global variable
def my_function():
    print(name)
my_function()
print(name)
```

This program outputs =>
Hello
Hello

```
def my_function():
    name = "Hello" #local variable
print(name)
```

This program raises an error
since the variable name is not
accessible outside the function.

Example

```
name = 'Mary'
```

```
def Greeting():
```

```
    name = 'Mary Poppins'
```

```
    print(name)
```



Mary Poppins
Mary

```
Greeting()
```

```
print(name)
```


Example

```
num = 6
```

```
def multi():
```

```
    global num
```

```
    num = num * 3
```

```
    print('Local variable:', num)
```

Local variable: 18

Global: 6

```
multi()
```

```
print('Global: ', num)
```

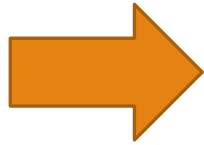
Best programming practices

Avoid use of global variables in favour of local variables.

Default Parameters

Default parameters are those arguments that take default values if no explicit values are passed to these arguments from the function call.

```
def greet(a,b,c,name="World"):
    print("Hello, " + name)
greet(2,3,4)
greet(2,3,4,"Abebe")
```



```
Hello, World
Hello, Abebe
```

Default parameters needs to be defined at the end of the parameter list. They shouldn't come before any parameters without default values.

Function calling: Positional

The method of calling is what we have been using through out the class. The first argument is for the first parameter, and the second argument is for the second parameter..

```
def maximum(x,y):  
    if(x > y):  
        return x  
    else:  
        return y  
max_val = maximum(2,4)  
print(max_val)
```

x = 2
y = 4

Function Calling: Keyword

In python, you can pass arguments with the *key = value* syntax. This way the arguments order doesn't matter.

```
def print_numbers(a,b,c):  
    print("a",a)  
    print("b",b)  
    print("c",c)  
print_numbers(b = 3,c = 2,a=1)
```



```
a 1  
b 3  
c 2
```

Functions: variable arguments

We can write functions that can accept variable length arguments like the built-in functions such as max and min.

```
def func(*args):  
    val = args[0]  
    for i in args:  
        print(i)  
func(1,2,3,4)  
func(1)  
func(1,2,3)
```

Demo: Let us write our own
max and min functions.
max(2,3,4,5)
max(2,3)